SERVICIO NACIONAL DE APRENDIZAJE - SENA

TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE

HOTEL GRAN MARQUES

Sistema de Gestión Hotelera

EVIDENCIA GA8-220501096-AA1-EV01

Desarrollar software a partir de la integración de sus módulos componentes

Aprendiz	Daniel Alejandro Ramirez Herrera
Identificación	C.C. 1036966567
Programa	Tecnología en Análisis y Desarrollo de Software (ADSO)
Ficha	3235881
Proyecto	Sistema de Gestión Hotelera - Hotel Gran Marques
Fecha	30 de octubre de 2026

Informe técnico de integración, codificación modular, configuración y pruebas

Tabla de contenido

- 1. Introducción y propósito de la evidencia
- 2. Descripción del proyecto, alcance y actores
- 3. Requisitos e historias de usuario
- 4. Arquitectura por capas y paquetes
- 5. Diagrama de componentes y modelo de dominio
- 6. Integración y codificación de módulos
- 7. Seguridad, persistencia y configuración de ambientes
- 8. IDE, tecnologías, librerías y metodología
- 9. Pruebas unitarias y de integración
- 10. Versionamiento Git y distribución
- 11. Conclusiones y limitaciones
- 12. Referencias

1. Introducción y propósito de la evidencia

La presente evidencia documenta la integración de los módulos que componen el Sistema de Gestión Hotelera del Hotel Gran Marques. El trabajo parte de la interfaz web desarrollada en HTML, CSS y JavaScript y de la API REST construida con Node.js. El propósito es conectar las vistas de operación con servicios de negocio, de modo que la información consultada y registrada provenga de un servidor común y se conserve en un repositorio de datos local para el ambiente académico.

La integración organiza el sistema en responsabilidades diferenciadas: presentación, cliente de servicios, API REST, módulos de dominio, persistencia y validación. Se documentan los requisitos, la arquitectura, las relaciones entre entidades, los controles de seguridad, la configuración de desarrollo y las pruebas automatizadas realizadas.

Objetivo general

Integrar y validar los componentes del sistema hotelero para que autenticación, huéspedes, habitaciones, reservas, consumos, facturación, pagos, mantenimiento y reportes interactúen mediante servicios REST y compartan una fuente de datos persistente.

Alcance

La evidencia cubre la integración funcional en un ambiente local de desarrollo. La persistencia se realiza mediante archivos JSON; por lo tanto, el proyecto es demostrativo y no se presenta como una instalación productiva o multiinstancia.

2. Descripción del proyecto, alcance y actores

Hotel Gran Marques es una aplicación web de gestión hotelera orientada al personal de recepción, administración y gerencia. Reúne operaciones de hospedaje y soporte: creación y consulta de huéspedes, disponibilidad de habitaciones, reservas, consumos adicionales, emisión de facturas, registro de abonos, solicitudes de mantenimiento y consolidación de indicadores.

Actor	Responsabilidades dentro del sistema
Recepción	Autenticarse, registrar huéspedes, crear/consultar reservas, registrar consumos y consultar facturación.
Administración	Gestionar catálogo de habitaciones y servicios, además de las funciones operativas permitidas.
Gerencia	Consultar indicadores consolidados de ocupación, reservas, facturación y mantenimiento.
Personal de mantenimiento	Consultar solicitudes y actualizar estado, prioridad y observaciones desde el módulo de mantenimiento.

Componentes existentes e integrados

La solución integra la interfaz (index.html, css/styles.css y js/app.js), el cliente HTTP y los servicios de negocio (js/api-client.js, js/services/ y js/domain-adapters.js), el servidor REST (api/server.js), la persistencia local (api/data/) y los componentes de calidad (tests/, scripts/, postman/ y docs/). La distribución de estas responsabilidades se detalla en los diagramas de capas y paquetes.

3. Requisitos e historias de usuario

ID	Requisito / historia	Módulo integrado
RF-01	Como usuario autorizado, quiero iniciar sesión y validar mi perfil.	Autenticación
RF-02	Como recepción, quiero registrar y consultar huéspedes.	Huéspedes
RF-03	Como recepción, quiero consultar habitaciones y disponibilidad.	Habitaciones
RF-04	Como recepción, quiero crear y actualizar reservas.	Reservas
RF-05	Como recepción, quiero asociar servicios adicionales a una reserva.	Servicios
RF-06	Como recepción, quiero generar una factura desde una reserva.	Facturación
RF-07	Como recepción, quiero registrar pagos parciales o totales.	Pagos
RF-08	Como mantenimiento, quiero crear y actualizar solicitudes.	Mantenimiento
RF-09	Como gerencia, quiero consultar indicadores del hotel.	Reportes

3.1 Requisitos no funcionales y trazabilidad

ID	Criterio de calidad	Aplicación en la solución
RNF-01	Seguridad	JWT Bearer en rutas protegidas, contraseñas con hash scrypt y permisos administrativos.
RNF-02	Integridad	Validación de campos, duplicados, capacidad, fechas, solapamiento de reservas y saldo de facturas.
RNF-03	Usabilidad	Menú constante, formularios con etiquetas, retroalimentación y estados visibles.
RNF-04	Mantenibilidad	Servicios de cliente y adaptadores separados del controlador de interfaz.
RNF-05	Interoperabilidad	HTTP y JSON; endpoints documentados en texto y OpenAPI.
RNF-06	Disponibilidad de desarrollo	Servidor local Node.js y pruebas reproducibles mediante scripts.

Matriz de trazabilidad requisito - componente - validación

Requisito	Componente UI	Servicio / endpoint	Validación
RF-01	Login	auth-service.js / POST /api/auth/login	Login válido e inválido
RF-02	Formulario de reserva (alta huésped)	hotel-service.js /api/guests	Crear y buscar huésped
RF-03	Habitaciones	/api/rooms	Consulta y regla de rol
RF-04	Reservas	/api/reservations	Crear, consultar y detectar cruce
RF-05	Servicios	/api/reservations/{id}/services	Agregar consumo
RF-06/RF-07	Facturación	/api/invoices y /payments	Factura única y abono
RF-08	Mantenimiento	/api/maintenance	Crear y actualizar estado
RF-09	Reportes	/api/reports/summary	Indicadores numéricos

La matriz permite rastrear cada función desde una necesidad del negocio hasta el componente que la consume y la prueba que comprueba su respuesta. Los endpoints y los ejemplos de estructuras JSON se encuentran en docs/API_REFERENCE.md, docs/ENDPOINTS_API.txt y docs/openapi.json.

4. Arquitectura por capas

La solución sigue una arquitectura web cliente-servidor. El navegador presenta la interfaz y delega las operaciones de negocio a servicios de cliente. Estos servicios usan un cliente HTTP común para invocar la API, que valida la sesión, aplica reglas del dominio y lee o actualiza el almacenamiento JSON.

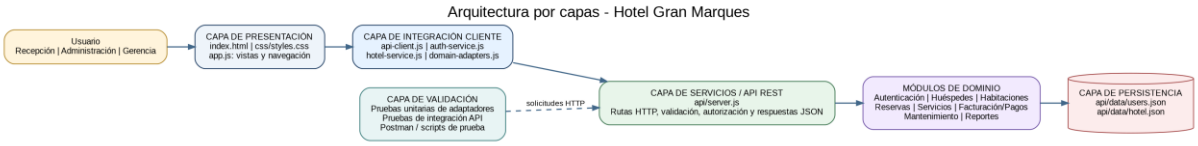


Figura 1. Arquitectura por capas de Hotel Gran Marques.

Capa	Responsabilidad	Archivos
Presentación	Estructura, estilos, accesibilidad visual y formularios.	index.html; css/styles.css
Control de interfaz	Navegación, eventos, renderizado y coordinación de vistas.	js/app.js
Integración cliente	Solicitudes, autenticación, servicios de hotel y mapeo.	js/api-client.js; js/services/*; js/domain-adapters.js
Servicios REST	Rutas HTTP, reglas, validación y autorización.	api/server.js
Persistencia	Datos de usuarios y datos hoteleros del ambiente local.	api/data/*.json
Calidad	Pruebas unitarias, integración y contratos.	tests/; scripts/; postman/; docs/

4.1 Diagrama de paquetes

El diagrama agrupa el código por responsabilidad física. En el cliente web se separan el controlador de interfaz, los adaptadores y los servicios que llaman a la API. El paquete servidor contiene el enrutador REST y los módulos de dominio; persistencia almacena los datos locales; calidad concentra pruebas, documentación y colección Postman.

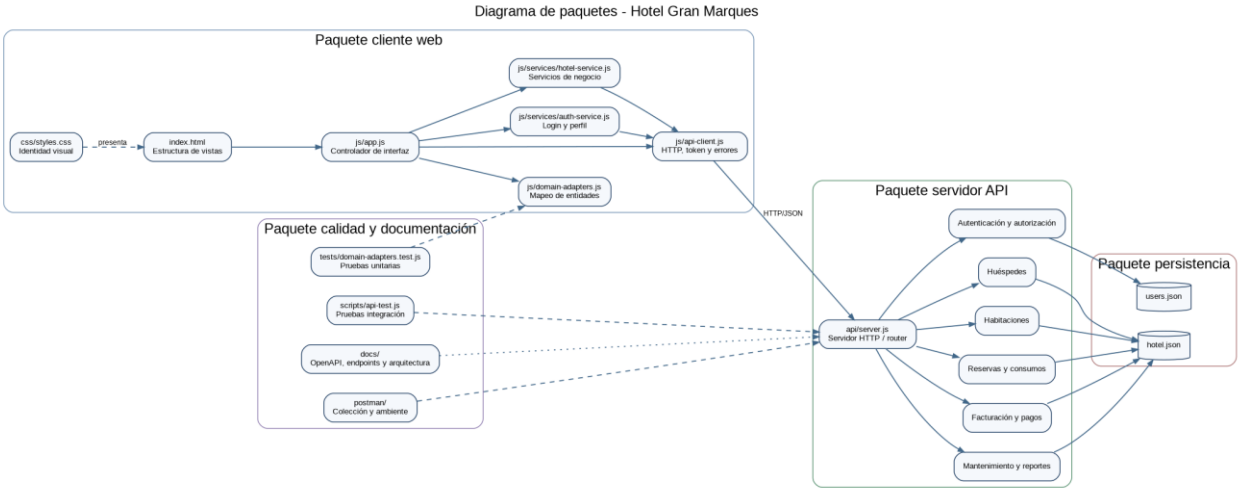


Figura 2. Diagrama de paquetes y dependencias principales.

Organización de carpetas

Ruta	Contenido
css/	Hoja de estilos compartida por las vistas.
js/services/	Servicios de autenticación y de operaciones hoteleras.
js/	Cliente HTTP, adaptadores y controlador de interfaz.
api/	Servidor HTTP Node.js y carpeta de persistencia local.
tests/	Pruebas unitarias de adaptadores de dominio.
scripts/	Pruebas de integración y proceso de distribución.
docs/diagrams/	Diagramas editables DOT y sus imágenes PNG.
postman/	Colección de solicitudes y ambiente local.

5. Diagrama de componentes y modelo de dominio

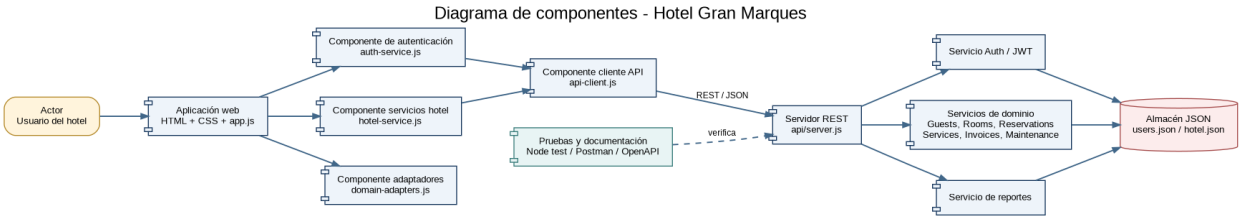


Figura 3. Interacción de componentes de cliente, API, dominio y calidad.

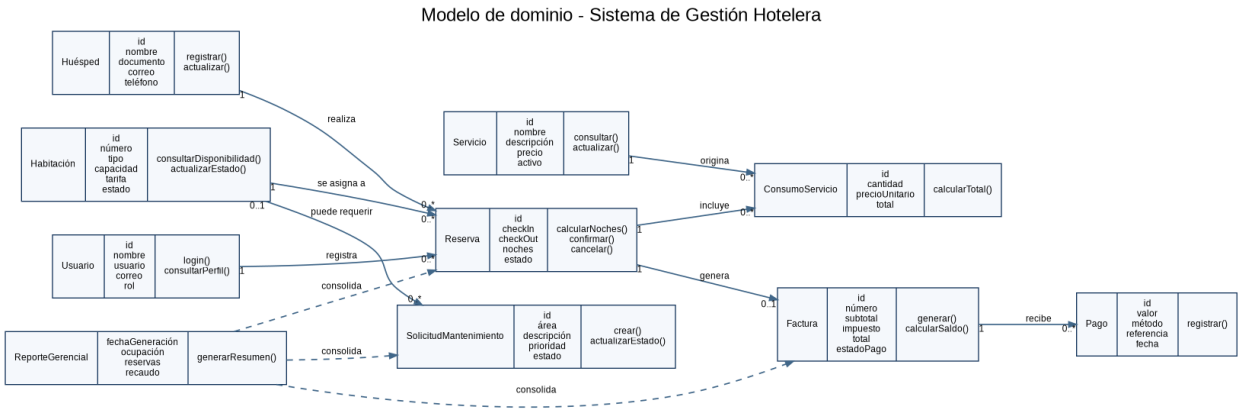


Figura 4. Modelo de dominio simplificado con asociaciones y multiplicidades.

Las asociaciones muestran que un huésped puede tener varias reservas y cada reserva se vincula con una habitación. Una reserva admite varios consumos y genera como máximo una factura; una factura admite varios pagos. Las habitaciones pueden asociarse con solicitudes de mantenimiento. Los reportes consolidan datos de reservas, facturación y mantenimiento.

6. Integración y codificación de módulos

El controlador de interfaz ya no gestiona registros hoteleros mediante arreglos persistidos en el navegador. Para las operaciones de negocio, solicita datos al servidor, procesa la respuesta y actualiza las vistas. El token de autenticación se agrega en el cliente HTTP y el servidor valida permisos según el rol.

Módulo	Operaciones integradas	Rutas REST
Autenticación	Login, perfil y token Bearer.	POST /api/auth/login; GET /api/auth/me
Huéspedes	Buscar, crear y consultar.	GET/POST /api/guests
Habitaciones	Listar, crear y actualizar (admin).	GET/POST/PATCH /api/rooms
Reservas	Listar, crear, consultar, actualizar/cancelar.	GET/POST/PATCH /api/reservations
Servicios	Listar catálogo, crear servicio (admin), agregar consumo.	GET/POST /api/services; POST /api/reservations/{id}/services
Facturas y pagos	Generar factura y registrar abonos.	POST /api/invoices; POST /api/invoices/{id}/payments
Mantenimiento	Crear solicitudes y actualizar estados.	GET/POST/PATCH /api/maintenance
Reportes	Consultar resumen gerencial.	GET /api/reports/summary

Secuencia general de una operación

1. El usuario selecciona una acción en la pantalla del módulo.
2. app.js valida datos básicos de presentación y llama a hotel-service.js o auth-service.js.
3. El servicio invoca api-client.js, que envía HTTP/JSON y añade Authorization cuando existe sesión.
4. api/server.js valida token, rol, campos y reglas de integridad antes de modificar datos.
5. El servidor devuelve código HTTP y objeto JSON; los adaptadores normalizan las entidades y la vista se vuelve a consultar.

Buenas prácticas aplicadas

- Separación entre transporte HTTP, servicios de negocio, adaptadores de datos y presentación.
- Nombres consistentes de funciones y archivos por responsabilidad.
- Validación de entradas también en el servidor; la validación visual no reemplaza la validación de API.
- Manejo centralizado de errores HTTP, estados de carga y mensajes para el usuario.

7. Seguridad, persistencia y configuración de ambientes

Seguridad

- Autenticación: el servidor emite un JWT firmado con HS256 y duración de una hora; las rutas de negocio requieren Bearer token.
- Contraseñas: se utiliza crypto.scrypt con sal aleatoria; el hash no se entrega en las respuestas públicas.
- Autorización: las operaciones administrativas de catálogo de habitaciones y servicios verifican el rol Administración.
- Integridad: se validan documentos duplicados, números de habitación, fechas, capacidad, solapamiento de reservas y límites de pago.
- Secretos: .env está excluido de Git; .env.example sirve como plantilla. La clave real debe ser privada y tener entropía suficiente.

El servidor actual permite CORS abierto para facilitar las pruebas locales; antes de desplegar en producción se debe restringir a los dominios autorizados, habilitar HTTPS, agregar limitación de intentos de login, auditoría y una gestión de secretos apropiada.

Persistencia y configuración

Elemento	Configuración del proyecto
Servidor	Node.js, módulo HTTP nativo, puerto 3000 por defecto.
Variables	PORT, NODE_ENV y JWT_SECRET, leídas desde .env o ambiente.
Usuarios	api/data/users.json, inicializado con cuentas locales de demostración.
Datos hoteleros	api/data/hotel.json, con huéspedes, habitaciones, reservas, servicios, facturas y mantenimiento.
Base de datos externa	No se requiere para el ambiente académico actual. Persistencia por archivos JSON.
Ambiente de pruebas	Mismo servidor local, scripts automatizados, colección Postman y datos generados por los tests.

Cuentas locales de demostración

Recepción: recepcion / 123456. Administración: admin / 123456. Son cuentas de práctica local; deben cambiarse o eliminarse en cualquier despliegue real.

8. IDE, tecnologías, librerías y metodología

Capa	Tecnología / herramienta	Uso
Presentación	HTML5, CSS3 y JavaScript nativo.	Estructura, estilos, eventos y actualización de vistas.
Cliente web	Fetch API del navegador.	Solicitudes HTTP a servicios REST.
Servidor	Node.js 18+; validación con Node.js 22 en el entorno de construcción y Node.js 24.22.1 indicado en el equipo de la aprendiz.	Servidor HTTP, lectura de archivos, criptografía y pruebas.
Persistencia	JSON y fs/promises.	Datos de prueba locales y actualización atómica de archivos.
Versionamiento	Git y GitHub.	Historial de cambios, ramas y publicación remota.
API / documentación	OpenAPI 3.0.3, Postman.	Descripción del contrato, endpoints y pruebas manuales.
Diagramas	Graphviz DOT.	Generación de diagramas de capas, paquetes, componentes y dominio.

IDE y entorno de trabajo

El proyecto puede editarse y depurarse en Visual Studio Code u otro IDE compatible con JavaScript. Las instrucciones de ejecución y versionamiento están preparadas para terminal Windows PowerShell o Git Bash. En el equipo de trabajo se verificó Node.js v24.22.1; el archivo package.json declara compatibilidad desde Node.js 18.

Librerías y frameworks

No se requieren dependencias npm externas. La solución utiliza módulos nativos de Node.js (http, fs/promises, path y crypto), JavaScript del navegador y Fetch API. No se incorpora un framework front-end como React o Angular ni un framework back-end; se prioriza una implementación ligera para facilitar el estudio de las capas y las llamadas HTTP.

Metodología

Se adopta una construcción incremental: se integra primero autenticación, luego módulos de operación y finalmente reportes y validaciones. Este enfoque permite entregar componentes verificables de manera progresiva. No se adjunta bitácora de sprints ni actas de Scrum, por lo que el documento describe el enfoque de trabajo y no afirma ceremonias que no están evidenciadas.

9. Pruebas unitarias y de integración

Se ejecutaron pruebas unitarias para funciones puras de adaptación de entidades y pruebas de integración HTTP contra un servidor Node.js local iniciado con los datos de demostración. Los resultados se guardaron en docs/RESULTADOS_GA8.txt y los resultados detallados de las solicitudes de integración en docs/resultado_pruebas_integracion_GA8.json/.txt.

Tipo	Comando	Aprobadas	Fallidas
Unitarias	npm run test:unit	6	0
Integración API	npm run test:integration	36	0
Total	npm test (servidor iniciado)	42	0

Cobertura de pruebas de integración

Área	Escenarios verificados
Salud y autenticación	Estado del servicio, login admin, perfil con token, registro, login nuevo e intento con clave errónea.
Huéspedes	Crear huésped, validar ID, buscar por documento.
Habitaciones	Listar habitaciones, verificar disponibilidad, bloquear alta a recepción y permitirla a administración.
Reservas	Crear reserva, calcular noches, impedir solapamiento y consultar por ID.
Servicios	Consultar catálogo activo y asociar consumo a una reserva.
Facturación y pagos	Generar factura, impedir duplicado y registrar abono parcial.
Mantenimiento y reportes	Crear solicitud, actualizar estado y consultar indicadores.
Seguridad	Rechazar consulta de negocio sin token.

Resultado de ejecución: 42 comprobaciones automatizadas aprobadas y ninguna fallida. La prueba automatizada no sustituye la aceptación funcional por el cliente ni una auditoría de seguridad de producción.

10. Versionamiento Git y distribución

El proyecto se integra en el repositorio GitHub utilizado para las evidencias anteriores. El enlace remoto proporcionado en los comandos de Git del proyecto es:

https://github.com/Alejandroramirez07/Gran_marques_frontend

Distribución y archivos ejecutables

El comando npm run build genera la carpeta dist/ con los archivos necesarios para ejecutar el sistema desde otra ubicación Node.js. Como el proyecto usa JavaScript nativo, no existe un paso de compilación a un binario .exe ni un bundle transpilado. En este contexto, dist/ es el artefacto de distribución ensamblado (HTML, CSS, JS, API y configuración) que se ejecuta mediante Node.js.

Estructura entregada

- Código fuente completo del sistema web y API.
- Carpeta dist/ con la distribución ejecutable.
- Pruebas unitarias e integración y resultados registrados.
- Documentación API, endpoints, OpenAPI y diagramas editables/PNG.
- Instrucciones de Git y enlace de repositorio.

11. Conclusiones y limitaciones

- La integración separa la interfaz del transporte HTTP y de los servicios de negocio, facilitando mantenimiento y cambios posteriores.
- Los módulos de huéspedes, habitaciones, reservas, servicios, facturación, pagos, mantenimiento y reportes se conectan a la misma API REST y persistencia local.
- La autenticación JWT, los roles y las validaciones de integridad protegen las rutas y los datos de operación.
- Se verificaron 6 pruebas unitarias y 36 pruebas de integración; las 42 comprobaciones registradas fueron aprobadas.
- La persistencia JSON es adecuada para el ambiente académico, pero debe migrarse a una base de datos transaccional antes de usar la solución en operación real.

12. Referencias

Alvarez Valencia, L. J. (2026). Informe técnico de endpoints, pruebas y guía de ejecución en Postman: Sistema de Gestión Hotelera Hotel Gran Marques [Documento de proyecto, evidencia GA7-220501096-AA5-EV02]. Servicio Nacional de Aprendizaje (SENA).

Git. (s. f.). Git documentation. https://git-scm.com/doc

MDN Web Docs. (s. f.). Fetch API. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Node.js. (s. f.). Node.js API documentation: HTTP, File system, Crypto and Test runner. https://nodejs.org/api/

OpenAPI Initiative. (2024). OpenAPI Specification, version 3.1.1. https://spec.openapis.org/oas/v3.1.1

OWASP Foundation. (s. f.). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Postman. (s. f.). Postman Learning Center. https://learning.postman.com/

Servicio Nacional de Aprendizaje [SENA]. (2026). Evidencia GA8-220501096-AA1-EV01: Desarrollar software a partir de la integración de sus módulos componentes [Guía de aprendizaje]. SENA.